



Maîtriser JavaScript & Vue.js — De Zéro à Pro

Documentation complète rédigée pour coder seul, sans IA, en toute autonomie.

TABLE DES MATIÈRES

PARTIE 1 — JAVASCRIPT FONDAMENTAL

1. [Variables & Types](#)
2. [Opérateurs](#)
3. [Structures de contrôle](#)
4. [Fonctions](#)
5. [Tableaux \(Arrays\)](#)
6. [Objets](#)
7. [Classes & POO](#)
8. [Destructuring & Spread](#)
9. [Modules \(import/export\)](#)
10. [Promesses & Async/Await](#)
11. [Gestion des erreurs](#)
12. [Manipulation du DOM](#)
13. [Événements](#)
14. [LocalStorage & SessionStorage](#)

PARTIE 2 — APPELS API REST

15. [HTTP & REST : les bases](#)
16. [fetch\(\) — Appels API](#)
17. [Axios](#)
18. [CRUD complet \(GET, POST, PUT, DELETE\)](#)
19. [Headers, Auth, Tokens](#)
20. [Gestion des erreurs API](#)

PARTIE 3 — VUE.JS FONDAMENTAL

- 21. [Introduction & Installation](#)
- 22. [Structure d'un composant Vue](#)
- 23. [Options API vs Composition API](#)
- 24. [Template Syntax](#)
- 25. [Réactivité : ref & reactive](#)
- 26. [Computed Properties](#)
- 27. [Watchers](#)
- 28. [Directives Vue](#)
- 29. [Événements & Méthodes](#)
- 30. [Formulaires & v-model](#)

PARTIE 4 — VUE.JS INTERMÉDIAIRE

- 31. [Composants & Props](#)
- 32. [Emit — Enfant vers Parent](#)
- 33. [Slots](#)
- 34. [Lifecycle Hooks](#)
- 35. [Vue Router](#)
- 36. [Pinia — State Management](#)

PARTIE 5 — VUE.JS + API REST

- 37. [Appels API dans Vue](#)
- 38. [Afficher des données depuis une API](#)
- 39. [Formulaire → POST vers une API](#)
- 40. [CRUD complet dans Vue](#)
- 41. [Authentification JWT dans Vue](#)
- 42. [Intercepteurs Axios dans Vue](#)

PARTIE 6 — PRO TIPS & PATTERNS

- 43. [Composables \(logique réutilisable\)](#)
 - 44. [Provide / Inject](#)
 - 45. [Transitions & Animations](#)
 - 46. [Optimisation & Performance](#)
 - 47. [Bonnes pratiques & Anti-patterns](#)
-
-

PARTIE 1 — JAVASCRIPT FONDAMENTAL

1. Variables & Types

Déclaration de variables

```
// var – ancienne syntaxe, évite-la (portée function, pas block)
var nom = "Alice";

// let – variable modifiable, portée block
let age = 25;
age = 26; // OK

// const – constante, ne peut pas être réassignée
const PI = 3.14;
// PI = 5; // ERREUR !
```

Règle d'or : Utilise toujours `const` . Si tu dois modifier la valeur, utilise `let` . N'utilise jamais `var` .

Types primitifs

```
// String (chaîne de caractères)
let prenom = "Marie";
let message = 'Bonjour';
let texte = `Bonjour ${prenom}`; // Template literal (interpolation)

// Number (entier et décimal)
let entier = 42;
let decimal = 3.14;
let negatif = -10;

// Boolean
let estActif = true;
let estAdmin = false;

// null – valeur intentionnellement vide
let utilisateur = null;

// undefined – variable déclarée mais pas assignée
let resultat;
console.log(resultat); // undefined

// Symbol – identifiant unique (avancé)
const id = Symbol("id");

// BigInt – très grands nombres
const grand = 999999999999999999n;
```

Vérifier le type

```
typeof "bonjour"    // "string"
typeof 42            // "number"
typeof true         // "boolean"
typeof undefined    // "undefined"
typeof null         // "object" ← bug historique JS !
typeof {}           // "object"
typeof []           // "object" ← array est un object !
typeof function(){} // "function"

// Pour vérifier un array :
Array.isArray([1, 2, 3]); // true
```

Conversion de types

```
// String vers Number
Number("42")        // 42
parseInt("42px")     // 42
parseFloat("3.14")  // 3.14
+"42"               // 42 (unaire +)

// Number vers String
String(42)           // "42"
(42).toString()     // "42"
42 + ""              // "42"

// Vers Boolean
Boolean(0)           // false
Boolean("")          // false
Boolean(null)        // false
Boolean(undefined)   // false
Boolean(NaN)         // false
Boolean("hello")     // true
Boolean(1)           // true
Boolean([])          // true ← attention !
Boolean({})          // true ← attention !
```

2. Opérateurs

Arithmétiques

```
5 + 3    // 8
5 - 3    // 2
5 * 3    // 15
5 / 2    // 2.5
5 % 2    // 1 (modulo – reste de la division)
5 ** 2   // 25 (puissance)
```

Comparaison

```
// == compare la valeur (avec conversion de type)
"5" == 5 // true ← DANGER !

// === compare la valeur ET le type (toujours utiliser ===)
"5" === 5 // false
5 === 5 // true

!= // différent (avec conversion)
!== // strictement différent (utilise toujours !==)

> >= < <= // comparaisons classiques
```

Logiques

```
true && false // false (ET – les deux doivent être vrais)
true || false // true (OU – au moins un vrai)
!true // false (NOT – inverse)

// Court-circuit (très utilisé en Vue)
const nom = utilisateur && utilisateur.nom; // si utilisateur existe, prend son nom
const valeur = donnee || "valeur par défaut"; // si donnee est falsy, prend la valeur par défaut
```

Nullish Coalescing (??)

```
// Différent de || : ne prend le fallback que si null ou undefined
const nom = utilisateur ?? "Anonyme";
// Si utilisateur = 0 → retourne 0 (pas "Anonyme")
// Si utilisateur = null → retourne "Anonyme"
```

Optional Chaining (?.)

```
// Évite les erreurs si l'objet est null/undefined
const ville = utilisateur?.adresse?.ville;
// Retourne undefined si utilisateur ou adresse est null/undefined
// Sans ?. : TypeError: Cannot read property 'ville' of undefined
```

Ternaire

```
const message = age >= 18 ? "Majeur" : "Mineur";
// condition ? valeur_si_vrai : valeur_si_faux
```

Assignation

```
let x = 10;
x += 5; // x = x + 5 → 15
x -= 3; // x = x - 3 → 12
x *= 2; // x = x * 2 → 24
x /= 4; // x = x / 4 → 6
x **= 2; // x = x ** 2 → 36
x ??= "default"; // x = x ?? "default"
```

3. Structures de contrôle

if / else if / else

```
const note = 75;

if (note >= 90) {
  console.log("Excellent");
} else if (note >= 70) {
  console.log("Bien");
} else if (note >= 50) {
  console.log("Passable");
} else {
  console.log("Insuffisant");
}
```

switch

```
const jour = "lundi";

switch (jour) {
  case "lundi":
  case "mardi":
    console.log("Début de semaine");
    break;
  case "vendredi":
    console.log("Fin de semaine");
    break;
  default:
    console.log("Milieu de semaine");
}
```

Boucle for classique

```
for (let i = 0; i < 5; i++) {
  console.log(i); // 0, 1, 2, 3, 4
}
```

Boucle while

```
let compteur = 0;
while (compteur < 3) {
  console.log(compteur);
  compteur++;
}
```

Boucle do...while (s'exécute au moins une fois)

```
let n = 0;
do {
  console.log(n);
  n++;
} while (n < 3);
```

for...of (parcourir un tableau)

```
const fruits = ["pomme", "banane", "cerise"];
for (const fruit of fruits) {
  console.log(fruit);
}
```

for...in (parcourir les clés d'un objet)

```
const personne = { nom: "Alice", age: 30 };
for (const cle in personne) {
  console.log(cle, personne[cle]);
  // nom Alice
  // age 30
}
```

4. Fonctions

Déclaration classique

```
function additionner(a, b) {
  return a + b;
}
additionner(3, 4); // 7
```

Expression de fonction

```
const multiplier = function(a, b) {
  return a * b;
};
multiplier(3, 4); // 12
```

Fonction fléchée (arrow function) — syntaxe moderne

```
// Forme complète
const diviser = (a, b) => {
  return a / b;
};

// Retour implicite (une seule expression)
const diviser = (a, b) => a / b;

// Un seul paramètre (pas besoin de parenthèses)
const double = n => n * 2;

// Sans paramètre
const saluer = () => "Bonjour!";
```

Paramètres par défaut

```
function saluer(nom = "Monde") {  
  return `Bonjour, ${nom}!`;  
}  
saluer("Alice"); // "Bonjour, Alice!"  
saluer();        // "Bonjour, Monde!"
```

Rest parameters (nombre variable d'arguments)

```
function somme(...nombres) {  
  return nombres.reduce((acc, n) => acc + n, 0);  
}  
somme(1, 2, 3, 4); // 10
```

Fonctions d'ordre supérieur (HOF)

```
// Une fonction qui reçoit une autre fonction en paramètre  
function appliquer(valeur, fn) {  
  return fn(valeur);  
}  
appliquer(5, n => n * 2); // 10  
  
// Une fonction qui retourne une fonction  
function multiplierPar(x) {  
  return (y) => x * y;  
}  
const triple = multiplierPar(3);  
triple(5); // 15
```

Fonctions de tableau (méthodes essentielles)

```
const nombres = [1, 2, 3, 4, 5];  
  
// map - transforme chaque élément, retourne un nouveau tableau  
nombres.map(n => n * 2); // [2, 4, 6, 8, 10]  
  
// filter - garde les éléments qui passent le test  
nombres.filter(n => n > 2); // [3, 4, 5]  
  
// reduce - réduit à une seule valeur  
nombres.reduce((acc, n) => acc + n, 0); // 15  
  
// find - retourne le premier élément qui passe le test  
nombres.find(n => n > 3); // 4  
  
// findIndex - retourne l'index du premier élément trouvé  
nombres.findIndex(n => n > 3); // 3  
  
// some - true si au moins un élément passe  
nombres.some(n => n > 4); // true  
  
// every - true si tous les éléments passent  
nombres.every(n => n > 0); // true  
  
// includes - vérifie la présence  
nombres.includes(3); // true  
  
// forEach - parcourt (ne retourne rien)  
nombres.forEach(n => console.log(n));
```


5. Tableaux (Arrays)

Création

```
const vide = [];  
const nombres = [1, 2, 3];  
const mixte = [1, "deux", true, null, { a: 1 }];  
const avecConstructeur = new Array(3).fill(0); // [0, 0, 0]
```

Accès et modification

```
const fruits = ["pomme", "banane", "cerise"];  
fruits[0]; // "pomme"  
fruits[fruits.length - 1]; // "cerise" (dernier élément)  
  
fruits[1] = "mangue"; // modifie l'élément
```

Méthodes de base

```
const arr = [1, 2, 3];  
  
// Ajouter / supprimer  
arr.push(4); // ajoute à la fin → [1,2,3,4]  
arr.pop(); // supprime la fin → [1,2,3]  
arr.unshift(0); // ajoute au début → [0,1,2,3]  
arr.shift(); // supprime le début → [1,2,3]  
  
// Splice – modifie en place  
arr.splice(1, 1); // supprime 1 élément à l'index 1  
arr.splice(1, 0, 99); // insère 99 à l'index 1  
arr.splice(1, 1, 99); // remplace l'élément à l'index 1 par 99  
  
// Slice – extrait sans modifier (retourne nouveau tableau)  
arr.slice(1, 3); // éléments index 1 à 2 (3 exclu)  
arr.slice(-2); // 2 derniers éléments  
  
// Concat – fusionne  
[1, 2].concat([3, 4]); // [1, 2, 3, 4]  
  
// Join – tableau vers string  
["a", "b", "c"].join("-"); // "a-b-c"  
  
// Reverse – inverse (modifie en place)  
[1, 2, 3].reverse(); // [3, 2, 1]  
  
// Sort – trie (modifie en place)  
["c", "a", "b"].sort(); // ["a", "b", "c"]  
[10, 2, 1].sort((a, b) => a - b); // [1, 2, 10] (numérique croissant)  
[10, 2, 1].sort((a, b) => b - a); // [10, 2, 1] (numérique décroissant)  
  
// Flat – aplatit les tableaux imbriqués  
[1, [2, [3]]].flat(); // [1, 2, [3]]  
[1, [2, [3]]].flat(Infinity); // [1, 2, 3]  
  
// FlatMap – map + flat(1)  
[[1], [2], [3]].flatMap(x => x); // [1, 2, 3]
```

6. Objets

Création

```
// Littéral (le plus courant)
const personne = {
  nom: "Alice",
  age: 30,
  saluer() {
    return `Bonjour, je m'appelle ${this.nom}`;
  }
};

// Accès aux propriétés
personne.nom;           // "Alice" (notation point)
personne["age"];        // 30 (notation crochet – utile pour les clés dynamiques)

// Modification
personne.age = 31;
personne.email = "alice@mail.com"; // ajoute une nouvelle propriété

// Suppression
delete personne.email;
```

Méthodes d'objet importantes

```
const obj = { a: 1, b: 2, c: 3 };

Object.keys(obj);      // ["a", "b", "c"]
Object.values(obj);    // [1, 2, 3]
Object.entries(obj);   // [["a", 1], ["b", 2], ["c", 3]]

// Créer un objet depuis des entrées
Object.fromEntries([["a", 1], ["b", 2]]); // { a: 1, b: 2 }

// Fusionner des objets
const obj2 = { d: 4 };
Object.assign({}, obj, obj2); // { a:1, b:2, c:3, d:4 }
// Ou avec spread (préféré) :
const fusion = { ...obj, ...obj2 }; // { a:1, b:2, c:3, d:4 }

// Vérifier si une propriété existe
"a" in obj;           // true
obj.hasOwnProperty("a"); // true
```

7. Classes & POO

```
class Animal {
  // Constructeur : appelé à la création de l'instance
  constructor(nom, age) {
    this.nom = nom;
    this.age = age;
  }

  // Méthode
  parler() {
    return `${this.nom} fait un bruit.`;
  }

  // Méthode statique (appelée sur la classe, pas l'instance)
  static creer(nom) {
    return new Animal(nom, 0);
  }

  // Getter
  get description() {
    return `${this.nom}, ${this.age} ans`;
  }

  // Setter
  set nouveauNom(nom) {
    if (typeof nom !== "string") throw new Error("Nom invalide");
    this.nom = nom;
  }
}

// Utilisation
const chat = new Animal("Minou", 3);
chat.parler(); // "Minou fait un bruit."
chat.description; // "Minou, 3 ans"
Animal.creer("Rex"); // nouvelle instance

// Héritage
class Chien extends Animal {
  constructor(nom, age, race) {
    super(nom, age); // appelle le constructeur parent
    this.race = race;
  }

  parler() {
    return `${this.nom} aboie!`; // surcharge la méthode parent
  }

  parlerCommeAnimal() {
    return super.parler(); // appelle la méthode du parent
  }
}

const rex = new Chien("Rex", 2, "Labrador");
rex.parler(); // "Rex aboie!"
rex instanceof Chien; // true
rex instanceof Animal; // true
```

8. Destructuring & Spread

Destructuring de tableau

```
const [premier, deuxieme, ...reste] = [1, 2, 3, 4, 5];
// premier = 1, deuxieme = 2, reste = [3, 4, 5]

// Valeur par défaut
const [a = 10, b = 20] = [5];
// a = 5, b = 20

// Ignorer des éléments
const [, troisieme] = [1, 2, 3];
// troisieme = 3

// Swap de variables
let x = 1, y = 2;
[x, y] = [y, x];
// x = 2, y = 1
```

Destructuring d'objet

```
const personne = { nom: "Alice", age: 30, ville: "Paris" };

const { nom, age } = personne;
// nom = "Alice", age = 30

// Renommer
const { nom: prenom, age: annees } = personne;
// prenom = "Alice", annees = 30

// Valeur par défaut
const { email = "inconnu@mail.com" } = personne;
// email = "inconnu@mail.com"

// Rest
const { nom: n, ...reste } = personne;
// n = "Alice", reste = { age: 30, ville: "Paris" }

// Destructuring dans les paramètres de fonction
function afficher({ nom, age }) {
  console.log(`${nom} a ${age} ans`);
}
afficher(personne);
```

Spread Operator (...)

```
// Tableau
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combine = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]

// Copie d'un tableau (shallow copy)
const copie = [...arr1];

// Objet
const obj1 = { a: 1, b: 2 };
const obj2 = { c: 3 };
const merge = { ...obj1, ...obj2 }; // { a: 1, b: 2, c: 3 }

// Modifier une propriété lors de la fusion
const updated = { ...obj1, b: 99 }; // { a: 1, b: 99 }

// Passer un tableau comme arguments
function somme(a, b, c) { return a + b + c; }
somme(...[1, 2, 3]); // 6
```

9. Modules (import/export)

Export

```
// utils.js

// Export nommé
export const PI = 3.14;
export function additionner(a, b) { return a + b; }
export class Calculatrice { /* ... */ }

// Export par défaut (un seul par fichier)
export default function principale() { /* ... */ }

// Export groupé
const a = 1;
const b = 2;
export { a, b };
export { a as alpha, b as beta }; // avec alias
```

Import

```
// Importer les exports nommés
import { PI, additionner } from "./utils.js";

// Avec alias
import { additionner as add } from "./utils.js";

// Importer tout sous un namespace
import * as Utils from "./utils.js";
Utils.additionner(1, 2);

// Importer l'export par défaut
import principale from "./utils.js";

// Mélanger
import principale, { PI, additionner } from "./utils.js";
```

10. Promesses & Async/Await

Promesse (Promise)

```
// Créer une promesse
const maPromesse = new Promise((resolve, reject) => {
  const succes = true;
  if (succes) {
    resolve("Données reçues!"); // succès
  } else {
    reject(new Error("Quelque chose a mal tourné")); // échec
  }
});

// Consommer une promesse
maPromesse
  .then(resultat => console.log(resultat)) // succès
  .catch(erreur => console.error(erreur)) // échec
  .finally(() => console.log("Terminé")); // toujours exécuté
```

Async/Await (syntaxe moderne — préférer)

```
// async devant une fonction = elle retourne toujours une Promise
async function chargerDonnees() {
  try {
    const reponse = await fetch("https://api.exemple.com/data");
    // await "attend" que la promesse se résolve
    const donnees = await reponse.json();
    return donnees;
  } catch (erreur) {
    console.error("Erreur:", erreur);
    throw erreur; // re-lancer l'erreur si nécessaire
  }
}

// Appel
chargerDonnees().then(data => console.log(data));

// Ou dans une autre fonction async :
async function main() {
  const data = await chargerDonnees();
  console.log(data);
}
```

Plusieurs promesses en parallèle

```
// Promise.all – attend que TOUTES se terminent (échoue si une échoue)
const [users, posts] = await Promise.all([
  fetch("/api/users").then(r => r.json()),
  fetch("/api/posts").then(r => r.json()),
]);

// Promise.allSettled – attend toutes, ne s'arrête pas aux erreurs
const resultats = await Promise.allSettled([promesse1, promesse2]);
resultats.forEach(r => {
  if (r.status === "fulfilled") console.log(r.value);
  if (r.status === "rejected") console.error(r.reason);
});

// Promise.race – prend le résultat de la PREMIÈRE qui se termine
const premier = await Promise.race([promesse1, promesse2]);

// Promise.any – prend le résultat du PREMIER succès
const unSucces = await Promise.any([promesse1, promesse2]);
```

11. Gestion des erreurs

```
// try / catch / finally
try {
  const resultat = JSON.parse("invalid json {{");
  console.log(resultat);
} catch (erreur) {
  console.error(erreur.name);    // "SyntaxError"
  console.error(erreur.message); // message d'erreur
  console.error(erreur.stack);   // trace complète
} finally {
  console.log("Ce bloc s'exécute toujours");
}

// Lancer ses propres erreurs
function diviser(a, b) {
  if (b === 0) throw new Error("Division par zéro impossible");
  return a / b;
}

// Types d'erreurs natifs
throw new TypeError("Mauvais type");
throw new RangeError("Hors limites");
throw new ReferenceError("Variable non définie");

// Classe d'erreur personnalisée
class ErreurAPI extends Error {
  constructor(message, statusCode) {
    super(message);
    this.name = "ErreurAPI";
    this.statusCode = statusCode;
  }
}

throw new ErreurAPI("Non trouvé", 404);
```

12. Manipulation du DOM

```
// Sélectionner des éléments
document.getElementById("monId");
document.querySelector(".maClasse"); // premier élément
document.querySelectorAll(".maClasse"); // tous les éléments (NodeList)
document.getElementsByClassName("cls"); // HTMLCollection

// Modifier le contenu
const el = document.querySelector("#titre");
el.textContent = "Nouveau titre"; // texte seulement (sécurisé)
el.innerHTML = "<strong>Gras</strong>"; // HTML (attention XSS!)

// Modifier les attributs
el.setAttribute("class", "ma-classe");
el.getAttribute("class");
el.removeAttribute("class");
el.classList.add("active");
el.classList.remove("active");
el.classList.toggle("active");
el.classList.contains("active");

// Modifier les styles
el.style.color = "red";
el.style.fontSize = "16px";
el.style.display = "none";

// Créer et ajouter des éléments
const div = document.createElement("div");
div.textContent = "Nouvel élément";
document.body.appendChild(div);
document.body.prepend(div); // avant tous les enfants
el.insertAdjacentHTML("afterend", "<p>Après</p>");

// Supprimer
el.remove();
el.parentNode.removeChild(el);

// Naviguer dans le DOM
el.parentElement;
el.children; // enfants HTML
el.childNodes; // tous les noeuds (texte inclus)
el.firstChild;
el.lastElementChild;
el.nextElementSibling;
el.previousElementSibling;
```


13. Événements

```
const bouton = document.querySelector("#mon-bouton");

// Ajouter un écouteur
bouton.addEventListener("click", function(event) {
  console.log("Cliqué!", event);
  event.preventDefault(); // empêche comportement par défaut (ex: soumission formulaire)
  event.stopPropagation(); // empêche la propagation (bubbling)
});

// Avec arrow function
bouton.addEventListener("click", (e) => {
  console.log(e.target); // l'élément qui a déclenché l'événement
  console.log(e.currentTarget); // l'élément sur lequel l'écouteur est attaché
});

// Supprimer un écouteur (nécessite une référence à la même fonction)
function handler(e) { console.log(e); }
bouton.addEventListener("click", handler);
bouton.removeEventListener("click", handler);

// Événements courants
// Souris
"click", "dblclick", "mousedown", "mouseup", "mousemove", "mouseenter", "mouseleave"
// Clavier
"keydown", "keyup", "keypress"
// Formulaire
"submit", "change", "input", "focus", "blur"
// Fenêtre
"load", "resize", "scroll", "beforeunload"
// Touch (mobile)
"touchstart", "touchend", "touchmove"

// Délégation d'événements (écouter sur le parent)
document.querySelector("#liste").addEventListener("click", (e) => {
  if (e.target.matches("li")) {
    console.log("Cliqué sur:", e.target.textContent);
  }
});
```

14. LocalStorage & SessionStorage

```
// localStorage – persiste après fermeture du navigateur
localStorage.setItem("cle", "valeur");
localStorage.getItem("cle"); // "valeur"
localStorage.removeItem("cle");
localStorage.clear(); // vide tout

// Stocker des objets (sérialisation JSON)
const utilisateur = { nom: "Alice", age: 30 };
localStorage.setItem("user", JSON.stringify(utilisateur));
const recupere = JSON.parse(localStorage.getItem("user"));

// sessionStorage – disparaît à la fermeture de l'onglet
sessionStorage.setItem("token", "abc123");

// Vérifier l'existence
const token = localStorage.getItem("token");
if (token) {
  // token existe
}
```

PARTIE 2 — APPELS API REST

15. HTTP & REST : les bases

Méthodes HTTP

Méthode	Usage	Body ?
GET	Récupérer des données	Non
POST	Créer une ressource	Oui
PUT	Remplacer une ressource	Oui
PATCH	Modifier partiellement	Oui
DELETE	Supprimer une ressource	Non

Codes de statut HTTP

Code	Signification
200	OK
201	Created (ressource créée)
204	No Content
400	Bad Request
401	Unauthorized (pas connecté)
403	Forbidden (pas autorisé)
404	Not Found
422	Unprocessable Entity
500	Internal Server Error

Structure d'une URL REST

`https://api.exemple.com/v1/users/42/posts?page=1&limit=10`

protocole domaine version resource id sous-ressource paramètres

Headers courants

Content-Type: application/json	(format du body envoyé)
Accept: application/json	(format attendu en retour)
Authorization: Bearer <token>	(authentification JWT)
Authorization: Basic <base64>	(authentification basique)

16. fetch() — Appels API

GET de base

```
async function getUtilisateurs() {
  const reponse = await fetch("https://jsonplaceholder.typicode.com/users");

  if (!reponse.ok) {
    throw new Error(`Erreur HTTP: ${reponse.status}`);
  }

  const donnees = await reponse.json();
  return donnees;
}
```

POST

```
async function creerUtilisateur(donnees) {
  const reponse = await fetch("https://api.exemple.com/users", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Accept": "application/json",
    },
    body: JSON.stringify(donnees),
  });

  if (!reponse.ok) {
    throw new Error(`Erreur: ${reponse.status}`);
  }

  return reponse.json();
}

creerUtilisateur({ nom: "Alice", email: "alice@mail.com" });
```

PUT / PATCH

```
async function modifierUtilisateur(id, donnees) {
  const reponse = await fetch(`https://api.exemple.com/users/${id}`, {
    method: "PUT", // ou "PATCH" pour modification partielle
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(donnees),
  });
  return reponse.json();
}
```

DELETE

```
async function supprimerUtilisateur(id) {
  const reponse = await fetch(`https://api.exemple.com/users/${id}`, {
    method: "DELETE",
  });

  if (!reponse.ok) throw new Error("Suppression échouée");
  // Souvent 204 No Content, pas de body JSON
  return true;
}
```

17. Axios

Axios est une bibliothèque qui simplifie les appels API. Elle gère automatiquement le JSON et offre plus d'options.

Installation

```
npm install axios
# ou CDN dans HTML
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
```

Utilisation de base

```
import axios from "axios";

// GET
const { data } = await axios.get("https://api.exemple.com/users");

// POST
const { data } = await axios.post("https://api.exemple.com/users", {
  nom: "Alice",
  email: "alice@mail.com",
});

// PUT
await axios.put(`https://api.exemple.com/users/${id}`, { nom: "Alice" });

// PATCH
await axios.patch(`https://api.exemple.com/users/${id}`, { nom: "Alice" });

// DELETE
await axios.delete(`https://api.exemple.com/users/${id}`);
```

Instance Axios (configuration centralisée)

```
// api.js
import axios from "axios";

const api = axios.create({
  baseURL: "https://api.exemple.com",
  timeout: 10000,
  headers: {
    "Content-Type": "application/json",
  },
});

export default api;

// Utilisation dans les autres fichiers :
import api from "./api";
const { data } = await api.get("/users");
const { data } = await api.post("/users", { nom: "Alice" });
```

18. CRUD complet

```
// service/userService.js - couche service (bonne pratique)
import api from "./api";

export const userService = {
  // Lire tous les utilisateurs
  async getAll() {
    const { data } = await api.get("/users");
    return data;
  },

  // Lire un utilisateur par ID
  async getById(id) {
    const { data } = await api.get(`/users/${id}`);
    return data;
  },

  // Créer un utilisateur
  async create(userData) {
    const { data } = await api.post("/users", userData);
    return data;
  },

  // Modifier complètement
  async update(id, userData) {
    const { data } = await api.put(`/users/${id}`, userData);
    return data;
  },

  // Modifier partiellement
  async patch(id, changes) {
    const { data } = await api.patch(`/users/${id}`, changes);
    return data;
  },

  // Supprimer
  async delete(id) {
    await api.delete(`/users/${id}`);
    return true;
  },
};
```

19. Headers, Auth, Tokens

Avec fetch

```
const token = localStorage.getItem("token");

const reponse = await fetch("https://api.exemple.com/profil", {
  headers: {
    "Authorization": `Bearer ${token}`,
    "Content-Type": "application/json",
  },
});
```

Avec Axios + intercepteur

```
// Ajouter automatiquement le token à chaque requête
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

20. Gestion des erreurs API

```
async function charger() {
  try {
    const { data } = await api.get("/users");
    return data;
  } catch (erreur) {
    if (erreur.response) {
      // Serveur a répondu avec un code d'erreur
      console.error("Status:", erreur.response.status);
      console.error("Data:", erreur.response.data);

      if (erreur.response.status === 401) {
        // Non authentifié → rediriger vers login
        router.push("/login");
      }
      if (erreur.response.status === 404) {
        console.error("Ressource non trouvée");
      }
    } else if (erreur.request) {
      // Requête envoyée mais pas de réponse (réseau ?)
      console.error("Pas de réponse du serveur");
    } else {
      // Erreur de configuration
      console.error("Erreur:", erreur.message);
    }
    throw erreur;
  }
}
```

PARTIE 3 — VUE.JS FONDAMENTAL

21. Introduction & Installation

Qu'est-ce que Vue.js ?

Vue.js est un framework JavaScript progressif pour construire des interfaces utilisateur. Il est réactif (les changements de données mettent automatiquement à jour l'interface) et basé sur des composants.

Installation avec Vite (recommandé)

```
npm create vue@latest mon-projet
cd mon-projet
npm install
npm run dev
```

Options lors de la création :

- TypeScript ? → Non (pour débiter)
- Vue Router ? → Oui
- Pinia ? → Oui
- ESLint ? → Oui

Structure d'un projet Vue

```
mon-projet/
├── public/           # Fichiers statiques
├── src/
│   ├── assets/      # Images, CSS globaux
│   ├── components/  # Composants réutilisables
│   ├── views/       # Pages (liées au router)
│   ├── stores/      # Pinia stores
│   ├── router/      # Configuration des routes
│   ├── App.vue      # Composant racine
│   └── main.js       # Point d'entrée
├── index.html
└── package.json
```

main.js (point d'entrée)

```
import { createApp } from "vue";
import { createPinia } from "pinia";
import App from "./App.vue";
import router from "./router";

const app = createApp(App);
app.use(createPinia());
app.use(router);
app.mount("#app");
```

22. Structure d'un composant Vue

Un composant Vue est un fichier `.vue` avec 3 sections :

```
<template>
  <!-- HTML du composant -->
  <div class="mon-composant">
    <h1>{{ titre }}</h1>
    <p>{{ message }}</p>
  </div>
</template>

<script setup>
// Logique JavaScript (Composition API)
import { ref } from "vue";

const titre = ref("Bonjour Vue!");
const message = ref("Mon premier composant.");
</script>

<style scoped>
/* CSS scopé à ce composant uniquement */
.mon-composant {
  color: navy;
}
</style>
```

23. Options API vs Composition API

Vue propose deux styles d'écriture. Dans cette doc, on privilégie la **Composition API** (moderne, plus puissante).

Options API (ancienne façon — toujours valide)

```
<script>
export default {
  data() {
    return {
      compteur: 0,
    };
  },
  computed: {
    double() {
      return this.compteur * 2;
    },
  },
  methods: {
    incrementer() {
      this.compteur++;
    },
  },
  mounted() {
    console.log("Composant monté!");
  },
};
</script>
```


Composition API avec `<script setup>` (moderne — à utiliser)

```
<script setup>
import { ref, computed, onMounted } from "vue";

const compteur = ref(0);
const double = computed(() => compteur.value * 2);

function incrementer() {
  compteur.value++;
}

onMounted(() => {
  console.log("Composant monté!");
});
</script>
```

24. Template Syntax

Interpolation de texte

```
<template>
  <p>{{ message }}</p>           <!-- Texte -->
  <p>{{ 1 + 2 }}</p>             <!-- Expression JS -->
  <p>{{ estActif ? "Oui" : "Non" }}</p> <!-- Ternaire -->
  <p>{{ message.toUpperCase() }}</p> <!-- Méthode -->
</template>
```

Binding d'attributs (v-bind)

```
<template>
  <!-- Lier un attribut à une variable -->
  

  <!-- Raccourci : (préférée) -->
  

  <!-- Classe dynamique -->
  <div :class="maClasse"></div>
  <div :class="{ active: estActif, disabled: !estActif }"></div>
  <div :class="['base', estActif ? 'active' : '']"></div>

  <!-- Style dynamique -->
  <p :style="{ color: couleur, fontSize: taille + 'px' }"></p>

  <!-- Attribut booléen -->
  <button :disabled="estDesactive">Cliquer</button>
</template>

<script setup>
import { ref } from "vue";
const imageUrl = ref("https://exemple.com/image.jpg");
const estActif = ref(true);
const couleur = ref("blue");
</script>
```

25. Réactivité : ref & reactive

ref — pour les valeurs primitives

```
<script setup>
import { ref } from "vue";

const compteur = ref(0);      // nombre
const nom = ref("Alice");    // string
const estActif = ref(false);  // boolean
const liste = ref([]);       // tableau

// Dans le script : utiliser .value
console.log(compteur.value);  // 0
compteur.value++;
compteur.value = 10;

// Dans le template : pas besoin de .value
// <p>{{ compteur }}</p> ← fonctionne directement
</script>

<template>
  <p>{{ compteur }}</p>
  <p>{{ nom }}</p>
</template>
```

reactive — pour les objets complexes

```
<script setup>
import { reactive } from "vue";

const utilisateur = reactive({
  nom: "Alice",
  age: 30,
  adresse: {
    ville: "Paris",
    pays: "France",
  },
});

// Accès direct (pas de .value !)
console.log(utilisateur.nom);
utilisateur.nom = "Bob";
utilisateur.adresse.ville = "Lyon";

// ATTENTION : ne pas réassigner l'objet entier !
// utilisateur = { nom: "Bob" }; // ← ERREUR, perd la réactivité
// Pour réinitialiser :
Object.assign(utilisateur, { nom: "Bob", age: 25 });
</script>

<template>
  <p>{{ utilisateur.nom }}</p>
  <p>{{ utilisateur.adresse.ville }}</p>
</template>
```

Quand utiliser ref vs reactive ?

ref → valeurs primitives (string, number, boolean), ou quand tu veux réassigner
reactive → objets avec plusieurs propriétés liées (formulaire, état d'une entité)

Conseil : En cas de doute, utilise toujours `ref`. C'est plus prévisible.

toRef & toRefs

```
import { reactive, toRefs } from "vue";

const state = reactive({ x: 0, y: 0 });

// Convertir les propriétés en refs individuelles
const { x, y } = toRefs(state);
// x.value et y.value restent liés à state
```

26. Computed Properties

Les computed sont des valeurs dérivées d'autres données réactives. Elles sont **mises en cache** et ne recalculent que si leurs dépendances changent.

```
<script setup>
import { ref, computed } from "vue";

const prenom = ref("Alice");
const nom = ref("Martin");
const prix = ref(100);
const taxe = ref(0.2);

// Computed simple
const nomCompleet = computed(() => `${prenom.value} ${nom.value}`);

// Computed avec calcul
const prixTTC = computed(() => prix.value * (1 + taxe.value));

// Computed avec filtrage
const items = ref([1, 2, 3, 4, 5]);
const itemsPairs = computed(() => items.value.filter(n => n % 2 === 0));

// Computed get/set (modifiable)
const nomCompleetRW = computed({
  get() {
    return `${prenom.value} ${nom.value}`;
  },
  set(valeur) {
    const parties = valeur.split(" ");
    prenom.value = parties[0];
    nom.value = parties[1] || "";
  },
});
</script>

<template>
  <p>{{ nomCompleet }}</p>
  <p>{{ prixTTC }}€</p>
</template>
```

27. Watchers

Les watchers permettent d'exécuter du code en réaction à des changements de données.

```

<script setup>
import { ref, watch, watchEffect } from "vue";

const recherche = ref("");
const compteur = ref(0);

// watch – surveille une ref spécifique
watch(recherche, (nouvelleValeur, ancienneValeur) => {
  console.log(`Changé de "${ancienneValeur}" à "${nouvelleValeur}"`);
  // Ici tu pourrais appeler une API de recherche
});

// watch avec options
watch(compteur, (val) => {
  console.log("Compteur:", val);
}, {
  immediate: true, // exécute immédiatement au montage
  deep: true,      // surveille les objets en profondeur
});

// Surveiller plusieurs sources
watch([recherche, compteur], ([nouvelleRecherche, nouveauCompteur]) => {
  console.log(nouvelleRecherche, nouveauCompteur);
});

// watchEffect – surveille toutes les dépendances utilisées
watchEffect(() => {
  // Vue détecte automatiquement les dépendances
  console.log("Recherche:", recherche.value);
  console.log("Compteur:", compteur.value);
});

// Stopper un watcher
const stop = watch(compteur, () => { /* ... */ });
stop(); // arrête le watcher
</script>

```

28. Directives Vue

v-if / v-else-if / v-else

```

<template>
  <div v-if="statut === 'charge'">Chargement...</div>
  <div v-else-if="statut === 'erreur'">Erreur !</div>
  <div v-else>Contenu chargé</div>
</template>

```

v-show

```

<!-- v-show change display:none, l'élément reste dans le DOM -->
<!-- Préférer v-show si l'élément alterne souvent -->
<!-- Préférer v-if si la condition est rarement vraie -->
<div v-show="estVisible">Je suis visible: {{ estVisible }}</div>

```

v-for

```
<template>
  <!-- Tableau de primitives -->
  <ul>
    <li v-for="(fruit, index) in fruits" :key="index">
      {{ index + 1 }}. {{ fruit }}
    </li>
  </ul>

  <!-- Tableau d'objets -->
  <div v-for="user in users" :key="user.id">
    {{ user.nom }} - {{ user.email }}
  </div>

  <!-- Objet -->
  <div v-for="(valeur, cle, index) in objet" :key="cle">
    {{ index }} - {{ cle }}: {{ valeur }}
  </div>

  <!-- Plage de nombres -->
  <span v-for="n in 5" :key="n">{{ n }} </span>
</template>

<script setup>
import { ref } from "vue";
const fruits = ref(["pomme", "banane", "cerise"]);
const users = ref([
  { id: 1, nom: "Alice", email: "alice@mail.com" },
  { id: 2, nom: "Bob", email: "bob@mail.com" },
]);
</script>
```

Important : Toujours utiliser `:key` avec `v-for`. Préfère un identifiant unique (id) plutôt que l'index.

v-bind (:)

```

<button :disabled="!formulaireValide">Envoyer</button>
<div :class="{ active: estActif }"></div>
```

v-on (@)

```
<button @click="incrimenter"></button>
<button @click="compteur++">Inline</button>
<input @input="e => valeur = e.target.value" />
<form @submit.prevent="envoyer">...</form>

<!-- Modificateurs -->
@click.stop      <!-- stopPropagation -->
@click.prevent   <!-- preventDefault -->
@click.once      <!-- écoute une seule fois -->
@keyup.enter     <!-- touche entrée -->
@keyup.ctrl.enter <!-- Ctrl + Entrée -->
```

v-model

```
<template>
  <input v-model="nom" />
  <!-- Équivalent à : -->
  <input :value="nom" @input="nom = $event.target.value" />
</template>
```

v-text & v-html

```
<p v-text="message"></p>      <!-- comme textContent -->
<div v-html="htmlContent"></div>  <!-- attention XSS ! -->
```

v-pre & v-once

```
<span v-pre>{{ Ceci ne sera pas compilé }}</span>
<div v-once>{{ Rendu une seule fois, pas réactif }}</div>
```

29. Événements & Méthodes

```
<template>
  <button @click="incrémenter">Incrémenter</button>
  <button @click="ajouter(5)">Ajouter 5</button>
  <button @click="(e) => gerer(e, 'extra')">Avec event</button>

  <!-- Modificateurs de touche -->
  <input @keyup.enter="soumettre" />
  <input @keyup.escape="annuler" />

  <!-- Modificateurs de souris -->
  <div @click.right.prevent="menuContextuel">Clic droit</div>
</template>

<script setup>
import { ref } from "vue";

const compteur = ref(0);

function incrémenter() {
  compteur.value++;
}

function ajouter(montant) {
  compteur.value += montant;
}

function gerer(event, extra) {
  console.log(event.target, extra);
}
</script>
```

30. Formulaires & v-model

```
<template>
  <form @submit.prevent="soumettre">
    <!-- Input texte -->
    <input v-model="form.nom" type="text" placeholder="Nom" />

    <!-- Input email -->
    <input v-model="form.email" type="email" />

    <!-- Textarea -->
    <textarea v-model="form.message"></textarea>

    <!-- Checkbox simple -->
    <input type="checkbox" v-model="form.accepte" />

    <!-- Checkboxes multiples (tableau) -->
    <input type="checkbox" value="a" v-model="form.options" />
    <input type="checkbox" value="b" v-model="form.options" />

    <!-- Radio -->
    <input type="radio" value="homme" v-model="form.genre" />
    <input type="radio" value="femme" v-model="form.genre" />

    <!-- Select -->
    <select v-model="form.pays">
      <option value="">Choisir</option>
      <option value="fr">France</option>
      <option value="mg">Madagascar</option>
    </select>

    <!-- Select multiple -->
    <select v-model="form.langues" multiple>
      <option value="fr">Français</option>
      <option value="en">Anglais</option>
    </select>

    <button type="submit">Envoyer</button>
  </form>
</template>

<script setup>
import { reactive } from "vue";

const form = reactive({
  nom: "",
  email: "",
  message: "",
  accepte: false,
  options: [],
  genre: "",
  pays: "",
  langues: [],
});

function soumettre() {
  console.log("Formulaire soumis:", form);
  // Appel API...
}
</script>
```

Modificateurs de v-model

```
<!-- .lazy – met à jour au changement (pas à chaque frappe) -->
<input v-model.lazy="texte" />

<!-- .number – convertit automatiquement en nombre -->
<input v-model.number="age" type="number" />

<!-- .trim – supprime les espaces en début/fin -->
<input v-model.trim="nom" />
```

PARTIE 4 — VUE.JS INTERMÉDIAIRE

31. Composants & Props

Créer un composant

```
<!-- components/CarteUtilisateur.vue -->
<template>
  <div class="carte">
    <h3>{{ nom }}</h3>
    <p>{{ email }}</p>
    <p v-if="age">{{ age }} ans</p>
    <slot />
  </div>
</template>

<script setup>
// Définition des props
const props = defineProps({
  nom: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    required: true,
    validator: (val) => val.includes("@"),
  },
  age: {
    type: Number,
    default: null,
  },
});

// Accéder aux props dans le script
console.log(props.nom);
</script>
```


Utiliser le composant

```
<!-- views/UtilisateursView.vue -->
<template>
  <div>
    <CarteUtilisateur
      nom="Alice"
      email="alice@mail.com"
      :age="30"
    />

    <!-- Passage dynamique -->
    <CarteUtilisateur
      v-for="user in users"
      :key="user.id"
      :nom="user.nom"
      :email="user.email"
    />

    <!-- Spread d'objet complet -->
    <CarteUtilisateur v-bind="user" />
  </div>
</template>

<script setup>
import CarteUtilisateur from "@components/CarteUtilisateur.vue";
import { ref } from "vue";

const users = ref([
  { id: 1, nom: "Alice", email: "alice@mail.com" },
  { id: 2, nom: "Bob", email: "bob@mail.com" },
]);
</script>
```

Types de props

```
defineProps({
  titre: String,
  age: Number,
  estActif: Boolean,
  liste: Array,
  config: Object,
  callback: Function,
  plusieurs: [String, Number], // plusieurs types
  requis: {
    type: String,
    required: true,
  },
  avecDefault: {
    type: String,
    default: "valeur par défaut",
  },
  objetDefault: {
    type: Object,
    default: () => ({ cle: "valeur" }), // factory function pour objets/tableaux
  },
  valide: {
    type: Number,
    validator: (val) => val >= 0 && val <= 100,
  },
});
```

32. Emit — Enfant vers Parent

```
<!-- components/BoutonCompteur.vue (enfant) -->
<template>
  <button @click="incrémenter">Compteur: {{ compteur }}</button>
</template>

<script setup>
import { ref } from "vue";

const compteur = ref(0);

// Déclarer les événements émis
const emit = defineEmits(["incrémenter", "reset"]);

function incrémenter() {
  compteur.value++;
  emit("incrémenter", compteur.value); // nom de l'événement + données
}
</script>
```

```
<!-- Parent -->
<template>
  <BoutonCompteur @incrémenter="surIncrement" />
</template>

<script setup>
import BoutonCompteur from "@components/BoutonCompteur.vue";

function surIncrement(valeur) {
  console.log("Nouvelle valeur:", valeur);
}
</script>
```

33. Slots

```
<!-- components/Carte.vue -->
<template>
  <div class="carte">
    <!-- Slot par défaut -->
    <slot>Contenu par défaut si aucun slot fourni</slot>

    <!-- Slot nommé -->
    <slot name="entete" />
    <slot name="pied" />

    <!-- Slot avec données (scoped slot) -->
    <slot name="item" :donnee="data" :index="0" />
  </div>
</template>
```

```

<!-- Utilisation -->
<template>
  <Carte>
    <!-- Slot par défaut -->
    <p>Mon contenu</p>

    <!-- Slot nommé -->
    <template #entete>
      <h2>Mon entête</h2>
    </template>

    <template #pied>
      <button>Action</button>
    </template>

    <!-- Scoped slot : récupérer les données du slot -->
    <template #item="{ donnee, index }">
      <p>{{ index }}: {{ donnee }}</p>
    </template>
  </Carte>
</template>

```

34. Lifecycle Hooks

```

<script setup>
import {
  onBeforeMount,
  onMounted,
  onBeforeUpdate,
  onUpdated,
  onBeforeUnmount,
  onUnmounted,
} from "vue";

// Avant que le composant soit inséré dans le DOM
onBeforeMount(() => {
  console.log("Avant montage");
});

// Composant inséré dans le DOM – idéal pour les appels API, accès au DOM
onMounted(() => {
  console.log("Composant monté! Appel API ici.");
});

// Avant une mise à jour du DOM suite à un changement de données
onBeforeUpdate(() => {
  console.log("Avant mise à jour");
});

// Après la mise à jour du DOM
onUpdated(() => {
  console.log("Après mise à jour");
});

// Avant que le composant soit retiré du DOM (nettoyage)
onBeforeUnmount(() => {
  console.log("Nettoyage...");
  // Annuler les timers, les écouteurs, les abonnements
});

// Composant retiré du DOM
onUnmounted(() => {
  console.log("Composant détruit");
});
</script>

```

Ordre d'exécution

1. setup() / <script setup>
2. onBeforeMount
3. Rendu du DOM
4. onMounted ← Idéal pour fetch API
5. (Changement de données)
6. onBeforeUpdate
7. (Mise à jour DOM)
8. onUpdated
9. (Destruction)
10. onBeforeUnmount
11. onUnmounted

35. Vue Router

Installation & Configuration

```
// router/index.js
import { createRouter, createWebHistory } from "vue-router";
import AccueilView from "@views/AccueilView.vue";

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: "/",
      name: "accueil",
      component: AccueilView,
    },
    {
      path: "/utilisateurs",
      name: "utilisateurs",
      // Chargement différé (lazy loading)
      component: () => import("@views/UtilisateursView.vue"),
    },
    {
      path: "/utilisateurs/:id",
      name: "utilisateur-detail",
      component: () => import("@views/UtilisateurDetailView.vue"),
    },
    {
      path: "/profil",
      component: () => import("@views/ProfilView.vue"),
      meta: { requiresAuth: true }, // metadata personnalisée
    },
    {
      path: "/*", // 404
      component: () => import("@views/NotFoundView.vue"),
    },
  ],
});

// Guard de navigation (protection de routes)
router.beforeEach((to, from) => {
  const estConnecte = !!localStorage.getItem("token");
  if (to.meta.requiresAuth && !estConnecte) {
    return { name: "login" };
  }
});

export default router;
```

Utilisation dans les templates

```
<template>
  <!-- Lien de navigation -->
  <RouterLink to="/">Accueil</RouterLink>
  <RouterLink :to="{ name: 'utilisateurs' }">Utilisateurs</RouterLink>
  <RouterLink :to="{ name: 'utilisateur-detail', params: { id: 42 } }">
    Profil
  </RouterLink>

  <!-- Active class automatique -->
  <RouterLink to="/accueil" active-class="actif">Accueil</RouterLink>

  <!-- Affichage de la vue courante -->
  <RouterView />
</template>
```

Utilisation dans le script

```
<script setup>
import { useRouter, useRoute } from "vue-router";

const router = useRouter(); // pour naviguer
const route = useRoute();   // pour lire les infos de la route courante

// Lire les paramètres
const id = route.params.id;
const page = route.query.page; // ?page=1

// Naviguer programmiquement
function allerAccueil() {
  router.push("/");
}

function allerProfil(id) {
  router.push({ name: "utilisateur-detail", params: { id } });
}

function allerRetour() {
  router.back();
}

function remplacer() {
  router.replace("/login"); // sans historique
}
</script>
```

36. Pinia — State Management

Pinia est le store officiel de Vue 3 pour partager l'état entre composants.

Créer un store

```
// stores/utilisateurStore.js
import { defineStore } from "pinia";
import { ref, computed } from "vue";
import { userService } from "@services/userService";

export const useUtilisateurStore = defineStore("utilisateur", () => {
  // State
  const utilisateurs = ref([]);
  const utilisateurCourant = ref(null);
  const enChargement = ref(false);
  const erreur = ref(null);

  // Getters (computed)
  const nombreUtilisateurs = computed(() => utilisateurs.value.length);
  const estConnecte = computed(() => utilisateurCourant.value !== null);

  // Actions
  async function chargerUtilisateurs() {
    enChargement.value = true;
    erreur.value = null;
    try {
      utilisateurs.value = await userService.getAll();
    } catch (e) {
      erreur.value = e.message;
    } finally {
      enChargement.value = false;
    }
  }

  async function ajouterUtilisateur(donnees) {
    const nouvel = await userService.create(donnees);
    utilisateurs.value.push(nouvel);
  }

  function seConnecter(user) {
    utilisateurCourant.value = user;
    localStorage.setItem("user", JSON.stringify(user));
  }

  function seDeconnecter() {
    utilisateurCourant.value = null;
    localStorage.removeItem("user");
    localStorage.removeItem("token");
  }

  return {
    utilisateurs,
    utilisateurCourant,
    enChargement,
    erreur,
    nombreUtilisateurs,
    estConnecte,
    chargerUtilisateurs,
    ajouterUtilisateur,
    seConnecter,
    seDeconnecter,
  };
});
```

Utiliser le store dans un composant

```
<script setup>
import { useUtilisateurStore } from "@/stores/utilisateurStore";
import { storeToRefs } from "pinia";
import { onMounted } from "vue";

const store = useUtilisateurStore();

// storeToRefs pour garder la réactivité lors du destructuring
const { utilisateurs, enChargement, erreur } = storeToRefs(store);

// Les actions n'ont pas besoin de storeToRefs
const { chargerUtilisateurs, ajouterUtilisateur } = store;

onMounted(() => {
  chargerUtilisateurs();
});
</script>

<template>
  <div v-if="enChargement">Chargement...</div>
  <div v-else-if="erreur">{{ erreur }}</div>
  <ul v-else>
    <li v-for="user in utilisateurs" :key="user.id">{{ user.nom }}</li>
  </ul>
</template>
```

PARTIE 5 — VUE.JS + API REST

37. Appels API dans Vue

Créer le service API

```
// services/api.js
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "https://api.exemple.com",
  timeout: 10000,
  headers: { "Content-Type": "application/json" },
});

// Intercepteur de requête (ajouter le token)
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Intercepteur de réponse (gestion globale des erreurs)
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);

export default api;
```

Variables d'environnement

```
# .env
VITE_API_URL=https://api.exemple.com

# .env.local (non commité, pour les secrets)
VITE_API_URL=http://localhost:3000
```

```
// Utilisation
const url = import.meta.env.VITE_API_URL;
```

38. Afficher des données depuis une API

```
<template>
  <div>
    <!-- État de chargement -->
    <div v-if="enChargement">Chargement...</div>

    <!-- État d'erreur -->
    <div v-else-if="erreur" class="erreur">
      {{ erreur }}
      <button @click="charger">Réessayer</button>
    </div>

    <!-- Données chargées -->
    <ul v-else>
      <li v-for="post in posts" :key="post.id">
        <h3>{{ post.title }}</h3>
        <p>{{ post.body }}</p>
      </li>
    </ul>
  </div>
</template>

<script setup>
import { ref, onMounted } from "vue";
import api from "@services/api";

const posts = ref([]);
const enChargement = ref(false);
const erreur = ref(null);

async function charger() {
  enChargement.value = true;
  erreur.value = null;
  try {
    const { data } = await api.get("/posts");
    posts.value = data;
  } catch (e) {
    erreur.value = e.response?.data?.message || "Erreur de chargement";
  } finally {
    enChargement.value = false;
  }
}

onMounted(charger);
</script>
```

39. Formulaire → POST vers une API

```
<template>
  <form @submit.prevent="creer">
    <div v-if="erreur" class="erreur">{{ erreur }}</div>
    <div v-if="succes" class="succes">Créé avec succès !</div>

    <input
      v-model="form.titre"
      type="text"
      placeholder="Titre"
      :disabled="enEnvoi"
    />
    <p v-if="erreursTitre" class="erreur-champ">{{ erreursTitre }}</p>

    <textarea v-model="form.contenu" :disabled="enEnvoi"></textarea>

    <button type="submit" :disabled="enEnvoi || !formulaireValide">
      {{ enEnvoi ? "Envoi..." : "Créer" }}
    </button>
  </form>
</template>

<script setup>
import { reactive, ref, computed } from "vue";
import api from "@services/api";

const form = reactive({ titre: "", contenu: "" });
const enEnvoi = ref(false);
const erreur = ref(null);
const succes = ref(false);

// Validation
const erreursTitre = computed(() => {
  if (!form.titre) return "Le titre est requis";
  if (form.titre.length < 3) return "Minimum 3 caractères";
  return null;
});

const formulaireValide = computed(() => !erreursTitre.value && form.contenu);

async function creer() {
  if (!formulaireValide.value) return;

  enEnvoi.value = true;
  erreur.value = null;
  succes.value = false;

  try {
    await api.post("/posts", {
      titre: form.titre,
      contenu: form.contenu,
    });
    succes.value = true;
    // Réinitialiser le formulaire
    form.titre = "";
    form.contenu = "";
  } catch (e) {
    erreur.value = e.response?.data?.message || "Erreur lors de la création";
  } finally {
    enEnvoi.value = false;
  }
}
</script>
```

40. CRUD complet dans Vue

```
<!-- views/PostsView.vue -->
<template>
  <div>
    <h1>Posts</h1>

    <!-- Formulaire de création/modification -->
    <form @submit.prevent="sauvegarder">
      <input v-model="form.titre" placeholder="Titre" />
      <textarea v-model="form.contenu"></textarea>
      <button type="submit">{{ modeEdition ? "Modifier" : "Créer" }}</button>
      <button v-if="modeEdition" type="button" @click="annulerEdition">
        Annuler
      </button>
    </form>

    <!-- Chargement -->
    <p v-if="enChargement">Chargement...</p>

    <!-- Liste -->
    <div v-for="post in posts" :key="post.id">
      <h3>{{ post.titre }}</h3>
      <p>{{ post.contenu }}</p>
      <button @click="editer(post)">Modifier</button>
      <button @click="supprimer(post.id)">Supprimer</button>
    </div>
  </div>
</template>

<script setup>
import { ref, reactive, onMounted } from "vue";
import api from "@services/api";

const posts = ref([]);
const enChargement = ref(false);
const modeEdition = ref(false);
const postEnEdition = ref(null);
const form = reactive({ titre: "", contenu: "" });

// Lire (Read)
async function charger() {
  enChargement.value = true;
  const { data } = await api.get("/posts");
  posts.value = data;
  enChargement.value = false;
}

// Créer ou Modifier (Create / Update)
async function sauvegarder() {
  if (modeEdition.value) {
    // Update
    const { data } = await api.put(`/posts/${postEnEdition.value.id}`, form);
    const index = posts.value.findIndex(p => p.id === data.id);
    posts.value[index] = data;
  } else {
    // Create
    const { data } = await api.post("/posts", form);
    posts.value.push(data);
  }
  annulerEdition();
}

// Préparer l'édition
function editer(post) {
  modeEdition.value = true;
  postEnEdition.value = post;
  form.titre = post.titre;
  form.contenu = post.contenu;
}

function annulerEdition() {
  modeEdition.value = false;
  postEnEdition.value = null;
  form.titre = "";
  form.contenu = "";
}
```

```
// Supprimer (Delete)
async function supprimer(id) {
  if (!confirm("Confirmer la suppression ?")) return;
  await api.delete(`/posts/${id}`);
  posts.value = posts.value.filter(p => p.id !== id);
}

onMounted(charger);
</script>
```

41. Authentification JWT dans Vue

```
// stores/authStore.js
import { defineStore } from "pinia";
import { ref, computed } from "vue";
import api from "@services/api";
import router from "@router";

export const useAuthStore = defineStore("auth", () => {
  const token = ref(localStorage.getItem("token") || null);
  const utilisateur = ref(
    JSON.parse(localStorage.getItem("user") || "null")
  );

  const estConnecte = computed(() => !!token.value);

  async function seConnecter(credentials) {
    const { data } = await api.post("/auth/login", credentials);
    token.value = data.token;
    utilisateur.value = data.user;
    localStorage.setItem("token", data.token);
    localStorage.setItem("user", JSON.stringify(data.user));
    router.push("/");
  }

  async function sInscrire(donnees) {
    const { data } = await api.post("/auth/register", donnees);
    token.value = data.token;
    utilisateur.value = data.user;
    localStorage.setItem("token", data.token);
    localStorage.setItem("user", JSON.stringify(data.user));
    router.push("/");
  }

  function seDeconnecter() {
    token.value = null;
    utilisateur.value = null;
    localStorage.removeItem("token");
    localStorage.removeItem("user");
    router.push("/login");
  }

  return { token, utilisateur, estConnecte, seConnecter, sInscrire, seDeconnecter };
});
```

```

<!-- views/LoginView.vue -->
<template>
  <form @submit.prevent="login">
    <input v-model="form.email" type="email" placeholder="Email" />
    <input v-model="form.password" type="password" placeholder="Mot de passe" />
    <p v-if="erreur">{{ erreur }}</p>
    <button type="submit" :disabled="enChargement">
      {{ enChargement ? "Connexion..." : "Se connecter" }}
    </button>
  </form>
</template>

<script setup>
import { reactive, ref } from "vue";
import { useAuthStore } from "@stores/authStore";

const authStore = useAuthStore();
const form = reactive({ email: "", password: "" });
const erreur = ref(null);
const enChargement = ref(false);

async function login() {
  enChargement.value = true;
  erreur.value = null;
  try {
    await authStore.seConnecter(form);
  } catch (e) {
    erreur.value = e.response?.data?.message || "Identifiants invalides";
  } finally {
    enChargement.value = false;
  }
}
</script>

```

42. Intercepteurs Axios dans Vue

```

// Intercepteur de réponse complet
api.interceptors.response.use(
  (response) => {
    // Toute réponse 2xx passe ici
    return response;
  },
  async (error) => {
    const originalRequest = error.config;

    // Token expiré → tenter de rafraîchir
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
      try {
        const refreshToken = localStorage.getItem("refreshToken");
        const { data } = await axios.post("/auth/refresh", { refreshToken });
        localStorage.setItem("token", data.token);
        originalRequest.headers.Authorization = `Bearer ${data.token}`;
        return api(originalRequest); // relancer la requête originale
      } catch (refreshError) {
        localStorage.clear();
        window.location.href = "/login";
      }
    }

    return Promise.reject(error);
  }
);

```

PARTIE 6 — PRO TIPS & PATTERNS

43. Composables

Les composables sont des fonctions qui encapsulent et réutilisent de la logique avec état.

```
// composables/useChargement.js
import { ref } from "vue";

export function useChargement(fn) {
  const enChargement = ref(false);
  const erreur = ref(null);
  const donnees = ref(null);

  async function executer(...args) {
    enChargement.value = true;
    erreur.value = null;
    try {
      donnees.value = await fn(...args);
    } catch (e) {
      erreur.value = e.response?.data?.message || e.message;
    } finally {
      enChargement.value = false;
    }
  }

  return { enChargement, erreur, donnees, executer };
}
```

```
// composables/useAPI.js (composable CRUD générique)
import { ref } from "vue";
import api from "@/services/api";

export function useAPI(endpoint) {
  const items = ref([]);
  const enChargement = ref(false);
  const erreur = ref(null);

  async function lireTout() {
    enChargement.value = true;
    try {
      const { data } = await api.get(endpoint);
      items.value = data;
    } catch (e) {
      erreur.value = e.message;
    } finally {
      enChargement.value = false;
    }
  }

  async function creer(donnees) {
    const { data } = await api.post(endpoint, donnees);
    items.value.push(data);
    return data;
  }

  async function modifier(id, donnees) {
    const { data } = await api.put(`${endpoint}/${id}`, donnees);
    const i = items.value.findIndex(item => item.id === id);
    if (i !== -1) items.value[i] = data;
    return data;
  }

  async function supprimer(id) {
    await api.delete(`${endpoint}/${id}`);
    items.value = items.value.filter(item => item.id !== id);
  }

  return { items, enChargement, erreur, lireTout, creer, modifier, supprimer };
}
```

```
<!-- Utilisation du composable -->
<script setup>
import { useAPI } from "@/composables/useAPI";
import { onMounted } from "vue";

const { items: posts, enChargement, lireTout, creer, supprimer } = useAPI("/posts");

onMounted(lireTout);
</script>
```

44. Provide / Inject

Permet de passer des données de n'importe quel ancêtre à n'importe quel descendant sans passer par chaque composant intermédiaire (évite le "prop drilling").

```
<!-- Composant ancêtre (App.vue ou layout) -->
<script setup>
import { provide, ref } from "vue";

const theme = ref("clair");
function basculerTheme() {
  theme.value = theme.value === "clair" ? "sombre" : "clair";
}

// Fournir la valeur
provide("theme", theme);
provide("basculerTheme", basculerTheme);
</script>
```

```
<!-- Composant descendant (peu importe la profondeur) -->
<script setup>
import { inject } from "vue";

const theme = inject("theme");
const basculerTheme = inject("basculerTheme");

// Avec valeur par défaut
const config = inject("config", { langue: "fr" });
</script>

<template>
  <div :class="theme">
    <button @click="basculerTheme">Changer le thème</button>
  </div>
</template>
```

45. Transitions & Animations

```
<template>
  <!-- Transition sur un élément unique -->
  <Transition name="fonde">
    <p v-if="estVisible">Contenu avec transition</p>
  </Transition>

  <!-- Transition de liste -->
  <TransitionGroup name="liste" tag="ul">
    <li v-for="item in items" :key="item.id">{{ item.nom }}</li>
  </TransitionGroup>
</template>

<style>
/* Fonde */
.fonde-enter-active,
.fonde-leave-active {
  transition: opacity 0.3s ease;
}

.fonde-enter-from,
.fonde-leave-to {
  opacity: 0;
}

/* Liste */
.liste-enter-active,
.liste-leave-active {
  transition: all 0.3s ease;
}

.liste-enter-from,
.liste-leave-to {
  opacity: 0;
  transform: translateX(-30px);
}

.liste-move {
  transition: transform 0.3s ease;
}
</style>
```

46. Optimisation & Performance

```
<script setup>
import { ref, shallowRef, markRaw, defineAsyncComponent } from "vue";

// shallowRef – réactivité en surface seulement (pour grands objets)
const grandeListeDeData = shallowRef([]);

// markRaw – exclure de la réactivité (bibliothèques tierces, instances Three.js...)
const carteLeaflet = markRaw(new Map());

// Composant chargé de façon asynchrone (code splitting)
const GrandComposant = defineAsyncComponent(() =>
  import("@/components/GrandComposant.vue")
);

// v-memo – mémorise un sous-arbre, ne re-rend que si les dépendances changent
</script>

<template>
  <!-- v-memo : ne re-render que si user.id ou user.actif change -->
  <div v-for="user in users" :key="user.id" v-memo="[user.id, user.actif]">
    {{ user.nom }}
  </div>
</template>
```

Bonnes pratiques de performance

- Utilise `:key` unique et stable sur les `v-for`
- Préfère `v-show` à `v-if` pour les éléments qui s'affichent/cachent souvent
- Utilise `defineAsyncComponent` pour les composants lourds
- Évite les computed complexes dans les templates (préfère les computed properties)
- Utilise `shallowRef` et `markRaw` pour les données volumineuses non réactives

47. Bonnes pratiques & Anti-patterns

✓ À FAIRE

```
<!-- 1. Toujours utiliser :key unique dans v-for -->
<li v-for="item in items" :key="item.id">

<!-- 2. Utiliser des noms de composants en PascalCase -->
<MonComposant />

<!-- 3. Séparer la logique en composables -->
const { items, charger } = useUtilisateurs();

<!-- 4. Gérer les états de chargement et d'erreur -->
<div v-if="enChargement">Chargement...</div>
<div v-else-if="erreur">{{ erreur }}</div>
<div v-else>{{ contenu }}</div>

<!-- 5. Valider les props -->
defineProps({
  titre: { type: String, required: true }
});

<!-- 6. Nommer les emits -->
const emit = defineEmits(["mise-a-jour", "suppression"]);

<!-- 7. Utiliser des services pour les appels API -->
// services/postService.js → usePostStore → composant
```

✗ À ÉVITER

```
// 1. Ne JAMAIS muter les props directement
props.titre = "Nouveau titre"; // ✗ ERREUR

// 2. Ne pas utiliser l'index comme key si la liste change
<li v-for="(item, i) in items" :key="i"> // ✗ Bug potentiel

// 3. Éviter la logique complexe dans le template
{{ users.filter(u => u.actif).map(u => u.nom).join(", ") }} // ✗ → computed!

// 4. Ne pas réassigner un reactive entier
const state = reactive({ a: 1 });
state = { a: 2 }; // ✗ perd la réactivité

// 5. Ne pas oublier .value avec ref dans le script
const compteur = ref(0);
compteur = 1; // ✗ doit être compteur.value = 1

// 6. Ne pas faire d'appels API directement dans le template
// ✗ <div>{{ await api.get('/users') }}</div>
// ✔ Appeler dans onMounted ou un store

// 7. Ne pas utiliser v-if et v-for sur le même élément
<li v-if="estVisible" v-for="item in items"> // ✗
// ✔ Wrapper ou computed filtré
```

Structure de projet recommandée

```
src/
├─ assets/
├─ components/
│   └─ ui/           # Composants UI réutilisables (Bouton, Modal, etc.)
│   └─ feature/      # Composants liés aux fonctionnalités
├─ composables/      # Logique réutilisable
├─ layouts/          # Layouts (Default, Auth, etc.)
├─ router/
│   └─ index.js
├─ services/
│   └─ api.js        # Instance Axios configurée
│   └─ userService.js
├─ stores/
│   └─ authStore.js
├─ utils/            # Fonctions utilitaires pures
├─ views/            # Pages (une par route)
├─ App.vue
└─ main.js
```

Annexe — Commandes essentielles

```
# Créer un projet
npm create vue@latest mon-projet

# Installer les dépendances
npm install

# Lancer en développement
npm run dev

# Build pour production
npm run build

# Prévisualiser le build
npm run preview

# Installer des packages
npm install axios
npm install pinia
npm install vue-router

# Désinstaller
npm uninstall axios
```

Annexe — Raccourcis de directives

Complet	Raccourci
v-bind:src	:src
v-on:click	@click
v-slot:nom	#nom

Annexe — Références rapides

Sujet	Utilisation
<code>ref()</code>	Données primitives réactives
<code>reactive()</code>	Objets réactifs
<code>computed()</code>	Valeurs dérivées mises en cache
<code>watch()</code>	Réagir aux changements
<code>onMounted()</code>	Code au montage (fetch API)
<code>onUnmounted()</code>	Nettoyage
<code>defineProps()</code>	Déclarer les entrées du composant
<code>defineEmits()</code>	Déclarer les événements émis
<code>useRouter()</code>	Navigation programmatique
<code>useRoute()</code>	Lire les params/query de la route
<code>defineStore()</code>	Créer un store Pinia
<code>storeToRefs()</code>	Destructurer un store avec réactivité
<code>provide/inject</code>	Injection de dépendances

Documentation rédigée pour une maîtrise autonome et complète de JavaScript et Vue.js 3. Version : Vue 3 + Composition API + Pinia + Vue Router 4

4

Automatic Zoom Guide Vue.js 3 — Complet Boucles · Conditions · Checkbox · FormData · Filtres · Formulaires dynamiques

N° Section 1 Introduction — structure d'un composant Vue 2 Boucles avec v-for 3 Conditions — v-if, v-else, v-show 4 Checkbox simple 5 Plusieurs Checkboxes 6 Tout sélectionner 7 FormData avec Vue 8 Suppression multiple 9 Filtres — filter / find / some 10 Recherche et vérification 11 Formulaire dynamique — lignes ajoutables 12 Référence — directives, API, opérateurs

1. Introduction — Structure d'un composant Vue

Vue.js 3 utilise la Composition API avec

n `ref()` crée une variable réactive. Dans le template Vue enlève le `.value` automatiquement. Dans le script il faut toujours écrire `titre.value` pour lire ou modifier.

Concept React Vue 3 Variable réactive `useState()` `ref()` / `reactive()` Boucle `.map()` `v-for` Condition `{cond &&}` `v-if` / `v-show` Événement `onClick` `@click` Liaison `input value` + `onChange` `v-model` Submit formulaire `e.preventDefault()` `@submit.prevent` 2. Boucles avec `v-for`

2.1 Tableau simple

Script + Template

2.2 Tableau d'objets

Script + Template

2.3 Répéter N fois / avec index

Script + Template

`n :key` est obligatoire dans `v-for`. Utilisez toujours un id unique. Vue l'utilise pour optimiser le rendu du DOM. 3. Conditions — `v-if`, `v-else`, `v-show`

3.1 `v-if` / `v-else`

Script + Template

3.2 `v-else-if` (plusieurs conditions)

Script + Template

3.3 `v-show` (reste dans le DOM, juste masqué)

Script + Template